Linux内核与应用层通信方式学习验证 — 开发日志

目录

- 1. 任务背景
- 2. 环境准备
- 3. 01-proc文件系统
- 4. 02-sysfs文件系统
- 5. 03-netlink套接字
- 6. 04-ioctl
- 7. 05-字符设备
- 8. 06-块设备
- 9. 07-mmap内存映射
- 10. 08-信号（Signals）
- 11. 09-inotify和fanotify
- 12. 命令速查表

1. 任务背景

学习 Linux 内核与应用层之间常见的通信方式，在 Ubuntu 18.04 虚拟机下编写程序验证每种方式的工作原理。

1.1 九种通信方式概览

编号	通信方式	内核角色	用户态交互方式	程序类型
01	proc文件系统	内核模块创建 `/proc/helloworld`	`cat /proc/helloworld`	内核模块
02	sysfs文件系统	内核模块创建 `/sys/kernel/my_sysfs_example`	`cat` + `echo` 读写	内核模块
03	Netlink套接字	内核模块监听 Netlink 并回复	用户程序发送/接收消息	内核模块 + 用户态
04	ioctl	字符设备驱动 + ioctl 接口	`ioctl()` 系统调用	内核模块 + 用户态
05	字符设备	`/dev/char_example` read/write	`echo` + `cat`	内核模块
06	块设备	`/dev/block_example` 扇区读写	`dd`	内核模块
07	mmap内存映射	内核内存映射到用户地址空间	`mmap()`	内核模块 + 用户态
08	信号	内核线程周期推送信号	`sigaction()` 接收	内核模块 + 用户态
09	inotify/fanotify	内核内置子系统推送文件事件	`read()` 读取事件	纯用户态

1.2 九种方式的关键差异

按数据方向分：

方向	涉及方式
用户主动读（用户→内核→用户）	proc、sysfs、字符设备、块设备、ioctl
双向通信（任意一端发起）	Netlink、mmap（建立映射后双向读写的零拷贝）
内核主动推送	信号、inotify/fanotify

按通信模型分：

模型	涉及方式	类比
文件伪装型	proc、sysfs、字符设备、块设备	把函数调用伪装成 `cat`/`echo`/`dd`
套接字型	Netlink	和 TCP/UDP 一样的 `sendmsg`/`recvmsg`
命令控制型	ioctl	一个 fd + 命令码，万能控制接口
共享内存型	mmap	内核把物理页"借"给用户态，之后零拷贝
事件通知型	信号、inotify/fanotify	用户睡觉，内核发生事情时叫醒你

按是否需要系统调用完成每次数据搬运分：

每次读写都要系统调用	映射一次后永不进内核
proc、sysfs、Netlink、ioctl、字符设备、块设备、信号、inotify/fanotify	**mmap**（唯一例外——`printf("%s", mapped_mem)` 不经过内核）

2. 环境准备

2.1 系统环境

- **操作系统**：Ubuntu 18.04
- **内核版本**：`4.15.0-213-generic`
- **工作目录**：`/home/tjn/commu/`

2.2 安装编译依赖

```
sudo apt update
sudo apt install build-essential linux-headers-$(uname -r)
```

2.3 两类程序

类型	编译方式	运行方式	涉及编号
内核模块（`.ko`）	`make`（调用内核构建系统）	`insmod` 加载 / `rmmod` 卸载	01~08

类型	编译方式	运行方式	涉及编号
纯用户态程序	`gcc -o xxx xxx.c`	`./xxx` 直接运行	03、04、07、08、09

内核模块编译需要 `linux-headers` 匹配当前内核版本，Makefile 通过 `/lib/modules/$(uname -r)/build` 定位内核源码。

---

3. 01-proc文件系统

3.1 通信原理

`/proc` 是内核提供的虚拟文件系统，内核模块通过 `proc_create()` 创建文件节点，用户态通过标准文件操作（`cat`）读取。内核在 `proc_read` 回调中用 `copy_to_user()` 将数据从内核空间拷到用户空间。

3.2 文件结构

```
01-proc文件系统/
├── Makefile          # obj-m += proc_example.o
└── proc_example.c    # 内核模块
```

3.3 关键代码解析

```c
/* 声明 read 回调，兼容 4.15 的 struct file_operations */
/**
 * @brief proc_read 读取 /proc/helloworld 文件
 * @param file 文件指针
 * @param usr_buf 用户空间缓冲区指针
 * @param count 读取字节数
 * @param pos 文件位置指针
 * @return ssize_t 读取的字节数
 */
static ssize_t proc_read(struct file *file, char __user *usr_buf,
                         size_t count, loff_t *pos)
{
    char buffer[128];
    int rv;

    if (*pos > 0)
        return 0;           /* EOF：已读完，不再返回数据 */

    rv = snprintf(buffer, sizeof(buffer), "%s", MESSAGE);
    if (copy_to_user(usr_buf, buffer, rv))  /* 内核空间 → 用户空间 */
        return -EFAULT;

    *pos += rv;
    return rv;
}
```

```
/* 4.15 必须用 file_operations，5.6+ 才引入 proc_ops */
static const struct file_operations proc_fops = {
    .owner = THIS_MODULE,
    .read  = proc_read,
};

/* insmod 时：创建 /proc/helloworld */
static int __init proc_init(void)
{
    proc_create(PROC_NAME, 0666, NULL, &proc_fops);
    return 0;
}

/* rmmod 时：删除 /proc/helloworld */
static void __exit proc_exit(void)
{
    remove_proc_entry(PROC_NAME, NULL);
}
```

- `*pos` 用于记录读取位置，避免 `cat` 多次调用 `read()` 时重复输出
- `copy_to_user` 必须用在内核→用户空间的数据传输，不能直接用 `memcpy`

3.4 关键API

`copy_to_user(to, from, n)`

```
unsigned long copy_to_user(void __user *to, const void *from, unsigned long n);
```

- **作用**：把内核空间数据拷贝到用户空间。返回未成功拷贝的字节数，0 表示全部拷贝成功。不能用 `memcpy` 替代——它不做地址空间校验。

`proc_create(name, mode, parent, proc_fops)`

```
struct proc_dir_entry *proc_create(const char *name, umode_t mode,
                                   struct proc_dir_entry *parent,
                                   const struct file_operations *proc_fops);
```

- **name**：文件名，如 `"helloworld"`
- **mode**：权限，如 `0666`
- **parent**：父目录，`NULL` 表示 `/proc/` 根目录
- **proc_fops**：文件操作表。Ubuntu 18.04 内核 4.15 必须用 `struct file_operations`（`proc_ops` 在 5.6 才引入）

`remove_proc_entry(name, parent)`

```
void remove_proc_entry(const char *name, struct proc_dir_entry *parent);
```

- **作用**：删除先前创建的 /proc 文件。如果模块卸载时不调用，文件变成悬空指针，再次 `cat` 会导致内核崩溃。

3.5 数据流

```
用户态                              内核态
──────                              ──────
$ cat /proc/helloworld
        │
        ▼  read() 系统调用
        │──────────────────────────►  proc_read(file, usr_buf, count, pos)
                                     │
                                    *pos == 0 → 继续
                                     ├─ snprintf(buffer, "Hello, World!\n")
                                     ├─ copy_to_user(usr_buf, buffer, 14)
                                     ├─ *pos += 14
                                     └─ return 14
        ◄──────────────────────────
        │  (read() 收到 14)
        │
        ▼  再调一次 read()
        │──────────────────────────►  proc_read(file, usr_buf, count, pos)
        *pos == 14 > 0               │
        ◄──────────────────────────  return 0  (EOF)
        │
$  显示: "Hello, World!"
```

3.6 编译与验证

```
root@ubuntu:/home/tjn/commu# mkdir 01-proc
root@ubuntu:/home/tjn/commu# cd 01-proc/
root@ubuntu:/home/tjn/commu/01-proc# vim proc_example.c
root@ubuntu:/home/tjn/commu/01-proc# vim Makefile
root@ubuntu:/home/tjn/commu/01-proc# make
make -C /lib/modules/4.15.0-213-generic/build M=/home/tjn/commu/01-proc modules
make[1]: Entering directory '/usr/src/linux-headers-4.15.0-213-generic'
  CC [M]  /home/tjn/commu/01-proc/proc_example.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC      /home/tjn/commu/01-proc/proc_example.mod.o
  LD [M]  /home/tjn/commu/01-proc/proc_example.ko
make[1]: Leaving directory '/usr/src/linux-headers-4.15.0-213-generic'
root@ubuntu:/home/tjn/commu/01-proc# insmod
proc_example.ko     proc_example.mod.o  proc_example.o      .tmp_versions/
root@ubuntu:/home/tjn/commu/01-proc# insmod proc_example.ko
root@ubuntu:/home/tjn/commu/01-proc# cat /proc/helloworld
Hello, World!
root@ubuntu:/home/tjn/commu/01-proc# rmmod proc_example
root@ubuntu:/home/tjn/commu/01-proc#
```

验证结果：`cat /proc/helloworld` 输出 `Hello, World!`，内核模块正确创建了 proc 文件并返回数据。

---

4. 02-sysfs文件系统

4.1 通信原理

`/sys` 是内核导出设备/驱动信息的虚拟文件系统。内核通过 `sysfs_create_file()` 在 `/sys/kernel/` 下创建可读写文件，用户态用 `cat`/`echo` 读写。与 proc 类似，但 sysfs 专为设备模型设计，每个文件严格对应一个属性值。

4.2 文件结构

```
02-sysfs文件系统/
├── Makefile           # obj-m += sysfs_example.o
└── sysfs_example.c    # 内核模块
```

4.3 关键代码解析

```c
/* 定义文件属性：可读可写，权限 0664 */
static struct kobj_attribute example_attribute =
    __ATTR(my_sysfs_example, 0664, example_show, example_store);

/* cat 时调用：返回当前值 */
static ssize_t example_show(struct kobject *kobj, struct kobj_attribute *attr,
                            char *buf)
{
    return sprintf(buf, "%s", example_str);
}

/* echo 写入时调用：更新值 */
static ssize_t example_store(struct kobject *kobj, struct kobj_attribute *attr,
                             const char *buf, size_t count)
{
    snprintf(example_str, sizeof(example_str), "%s", buf);
    return count;
}

/* 直接在 /sys/kernel/ 下创建文件，不新建子目录 */
static int __init sysfs_example_init(void)
{
    error = sysfs_create_file(kernel_kobj, &example_attribute.attr);
    return error;
}
```

- `__ATTR(name, mode, show, store)` 宏一次性定义属性名、权限、读写回调

- `kernel_kobj` 是内核全局 kobject，代表 `/sys/kernel/` 目录

4.4 关键API

`sysfs_create_file(kobj, attr)` / `sysfs_remove_file(kobj, attr)`

```
int sysfs_create_file(struct kobject *kobj, const struct attribute *attr);
void sysfs_remove_file(struct kobject *kobj, const struct attribute *attr);
```

- **kobj**：父 kobject，文件创建在其 sysfs 目录下
- **attr**：指向 `kobj_attribute.attr`，即通过 `__ATTR` 宏定义的属性

`__ATTR(name, mode, show, store)`

```
#define __ATTR(_name, _mode, _show, _store) { ... }
```

- **name**：sysfs 文件名
- **mode**：权限（如 `0664` 表示 owner/group 可读写，other 只读）
- **show**：读回调函数指针（`cat` 时触发）
- **store**：写回调函数指针（`echo` 时触发）
- **注意**：`show` 和 `store` 的 `buf` 参数都是内核空间地址，**不需要** `copy_to_user`/`copy_from_user`，sysfs 框架帮你做了

`kernel_kobj`

- 内核全局变量（`extern struct kobject *kernel_kobj`），代表 `/sys/kernel/` 目录
- 直接在它上面 `sysfs_create_file(kernel_kobj, &attr)` 就创建 `/sys/kernel/xxx`

4.5 数据流

```
用户态                                  内核态
──────                                  ──────
$ echo "Hi" > /sys/kernel/my_sysfs_example
        │
        ▼  write() 系统调用
        ├──────────────────────────►  sysfs 框架: copy_from_user → buf
                                      │
                                      ▼
                                    example_store(kobj, attr, buf, count)
                                      ├─ snprintf(example_str, "%s", buf)
                                      └─ return count

$ cat /sys/kernel/my_sysfs_example
        │
        ▼  read() 系统调用
        ├──────────────────────────►  sysfs 框架
                                      │
```

```
                                    ▼
                                  example_show(kobj, attr, buf)
                                   ├─ sprintf(buf, "%s", example_str)
                                   └─ return len
                                   │
          ◄──────────────────────── sysfs 框架: copy_to_user(usr_buf, buf, len)
          │
$  显示: "Hi"
```

4.6 编译与验证

```
root@ubuntu:/home/tjn/commu/02-sysfs# insmod sysfs_example.ko
root@ubuntu:/home/tjn/commu/02-sysfs# cat /sys/kernel/my_sysfs_example
Hello, World!
root@ubuntu:/home/tjn/commu/02-sysfs# rmmod sysfs_example
```

验证结果：`cat /sys/kernel/my_sysfs_example` 正确输出 `Hello, World!`。

4.7 遇到的问题

问题：`kobject_create_and_add("kernel", kernel_kobj)` 在 `/sys/kernel/` 下创建了子目录 `kernel/`，导致文件路径变成 `/sys/kernel/kernel/my_sysfs_example`，与预期不符。

原因：`kobject_create_and_add("kernel", kernel_kobj)` 以 `kernel_kobj` 为父节点创建名为 `"kernel"` 的新 kobject，相当于 `/sys/kernel/kernel/`。

解决：直接用 `sysfs_create_file(kernel_kobj, &attr)` 在 `/sys/kernel/` 下创建文件，不新建中间 kobject。

---

5. 03-netlink套接字

5.1 通信原理

Netlink 是专为内核-用户态通信设计的套接字协议。内核模块创建 Netlink socket 并注册接收回调；用户态程序用标准 socket API（`socket()` / `bind()` / `sendmsg()` / `recvmsg()`）与内核交换消息。支持异步双向通信，广泛用于网络配置（如 `ip route` 命令底层就是 Netlink）。

5.2 文件结构

```
03-netlink套接字/
├── Makefile              # obj-m += netlink_example.o
├── netlink_example.c     # 内核模块
└── netlink_user.c        # 用户态程序
```

5.3 关键代码解析

内核模块 — 接收消息并回复：

```
/* cfg.input 注册回调，收到用户消息时自动调用 */
static struct netlink_kernel_cfg cfg = {
    .input = nl_recv_msg,
};
nl_sk = netlink_kernel_create(&init_net, NETLINK_USER, &cfg);

static void nl_recv_msg(struct sk_buff *skb)
{
    /* 从 skb 中提取发送方 PID */
    pid = nlh->nlmsg_pid;

    /* 构造回复 skb */
    skb_out = nlmsg_new(msg_size, GFP_KERNEL);
    nlh = nlmsg_put(skb_out, 0, 0, NLMSG_DONE, msg_size, 0);
    memcpy(nlmsg_data(nlh), msg, msg_size);

    /* 单播回复给发送方 */
    nlmsg_unicast(nl_sk, skb_out, pid);
}
```

用户态 — 发送并接收：

```
sock_fd = socket(PF_NETLINK, SOCK_RAW, NETLINK_USER);

/* bind 本机地址 */
src_addr.nl_pid = getpid();
bind(sock_fd, (struct sockaddr *)&src_addr, sizeof(src_addr));

/* 目标地址：pid=0 表示发往内核 */
dest_addr.nl_pid = 0;

/* NLMSG_LENGTH 不含对齐填充，避免 ENOBUFS */
nlh->nlmsg_len = NLMSG_LENGTH(strlen("Hello from user space") + 1);

sendmsg(sock_fd, &msg, 0);
recvmsg(sock_fd, &msg, 0);   /* 收到 "Hello from kernel" */
```

5.4 关键API

内核端

`netlink_kernel_create(&init_net, unit, &cfg)`

```
struct sock *netlink_kernel_create(struct net *net, int unit,
                                   struct netlink_kernel_cfg *cfg);
```

- **net**：网络命名空间，几乎总是 `&init_net`
- **unit**：协议编号，必须和用户态 `socket()` 第三个参数一致
- **cfg.input**：收到用户消息时的回调函数
- **cfg.groups**：多播组数量上限，不需要多播则填 0

`nlmsg_new(payload, gfp)`

```
struct sk_buff *nlmsg_new(int payload, gfp_t gfp);
```

- 分配一个 skb，`payload` 是载荷大小（不含 Netlink 头部本身）
- 返回的 skb 已预留了 `NLMSG_HDRLEN` 的空间给 `nlmsg_put` 使用

`nlmsg_put(skb, portid, seq, type, payload, flags)`

```
struct nlmsghdr *nlmsg_put(struct sk_buff *skb, u32 portid, u32 seq,
                           int type, int payload, int flags);
```

- 在 skb 里"盖上" Netlink 头部，返回的 `nlh` 指针指向头部起始位置
- **type**：消息类型，单条回复用 `NLMSG_DONE`
- **payload**：载荷字节数，必须和 `nlmsg_new` 的参数一致

`nlmsg_data(nlh)`

```
void *nlmsg_data(const struct nlmsghdr *nlh);
```

- 返回载荷区起始地址，等价于 `(char*)nlh + NLMSG_HDRLEN`（跳过 16 字节头）

`nlmsg_unicast(sk, skb, portid)`

```
int nlmsg_unicast(struct sock *sk, struct sk_buff *skb, u32 portid);
```

- 单播发送 skb 给 portid（用户进程 PID）
- 成功后 skb 归内核所有，**不能再手动 free**

用户态

`socket(PF_NETLINK, SOCK_RAW, NETLINK_USER)`

- 创建 Netlink socket，协议编号和内核一致

`NLMSG_LENGTH(len)` **vs** `NLMSG_SPACE(len)`

宏	公式	用途

宏	公式	用途
`NLMSG_LENGTH(len)`	`NLMSG_HDRLEN + len`	填入 `nlmsg_len`
`NLMSG_SPACE(len)`	`NLMSG_ALIGN(NLMSG_LENGTH(len))`	`malloc` 分配缓冲区大小

`sendmsg(sock_fd, &msg, 0)` / `recvmsg(sock_fd, &msg, 0)`

- 发送时 `iov.iov_len` = `nlh->nlmsg_len`（实际消息长度）
- 接收时 `iov.iov_len` = `NLMSG_SPACE(MAX_PAYLOAD)`（缓冲区容量）
- 目标地址 `nl_pid=0` 表示发给内核

5.5 数据流

```
用户态                                    内核态
──────                                    ──────
socket(PF_NETLINK, SOCK_RAW, 31)
bind(nl_pid=getpid())                     netlink_kernel_create(31, cfg)
  │                                         │
  │  sendmsg(dest_pid=0)                    │
  ├──"Hello from user space"──►          nl_recv_msg(skb)
  │                                         │
  │                                         ├─ pid = nlh->nlmsg_pid
  │                                         ├─ printk("received: ...")
  │                                         ├─ nlmsg_new("Hello from kernel")
  │                                         └─ nlmsg_unicast(→ pid)
  │                                           │
  ◄──────── recvmsg() ◄─────────────────────┘
  │
printf("Received: Hello from kernel")
```

5.6 编译与验证

```
root@ubuntu:/home/tjn/commu/03-netlink# vim netlink_user.c
root@ubuntu:/home/tjn/commu/03-netlink# vim netlink_example.c
root@ubuntu:/home/tjn/commu/03-netlink# make
make -C /lib/modules/4.15.0-213-generic/build M=/home/tjn/commu/03-netlink modules
make[1]: Entering directory '/usr/src/linux-headers-4.15.0-213-generic'
  CC [M]  /home/tjn/commu/03-netlink/netlink_example.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC      /home/tjn/commu/03-netlink/netlink_example.mod.o
  LD [M]  /home/tjn/commu/03-netlink/netlink_example.ko
make[1]: Leaving directory '/usr/src/linux-headers-4.15.0-213-generic'
root@ubuntu:/home/tjn/commu/03-netlink# insmod netlink_example.ko
root@ubuntu:/home/tjn/commu/03-netlink# gcc -o netlink_user netlink_user.c
root@ubuntu:/home/tjn/commu/03-netlink# ./netlink_user
Sent message to kernel, waiting for reply...
Received message payload: Hello from kernel
root@ubuntu:/home/tjn/commu/03-netlink# rmmod netlink_example
```

验证结果：用户态发 `"Hello from user space"`，收到内核回复 `"Hello from kernel"`，双向通信成功。

5.7 遇到的问题

问题 1：`sendmsg: No buffer space available`（ENOBUFS）。

原因：`nlmsg_len` 用 `NLMSG_SPACE(MAX_PAYLOAD)` 而非 `NLMSG_LENGTH(strlen(msg)+1)`。`NLMSG_SPACE` 包含对齐填充，导致内核认为消息长度超过实际内容，socket 缓冲区校验失败。

解决：先 `strcpy` 写数据，再 `nlh->nlmsg_len = NLMSG_LENGTH(strlen("Hello from user space") + 1)`。

问题 2：`recvmsg` 收不到回复或收到乱码。

原因：`recvmsg` 复用同一个 `nlh` 缓冲区但未重新初始化 `iov`，且内核模块用 `strncpy` 未加 `\0`。

解决：

- 用户态：`recvmsg` 前 `memset(nlh, 0, NLMSG_SPACE(MAX_PAYLOAD))` + 重新设置 `iov`
- 内核态：`memcpy(nlmsg_data(nlh), msg, msg_size)` + `msg_size = strlen(msg) + 1`（含空字符）
- 增加 `setsockopt` 调大 `SO_SNDBUF` / `SO_RCVBUF` 到 256KB

---

6. 04-ioctl

6.1 通信原理

`ioctl`（input/output control）是设备驱动的万能控制接口。内核模块注册为字符设备并实现 `unlocked_ioctl` 回调；用户态通过 `ioctl(fd, cmd, arg)` 传入命令码和参数指针，内核根据命令码执行对应操作（读/写/配置）。

6.2 文件结构

```
04-ioctl/
├── Makefile          # obj-m += ioctl_example.o
├── ioctl_example.c   # 内核模块（字符设备 + ioctl）
└── ioctl_user.c      # 用户态程序
```

6.3 关键代码解析

```
/* 定义 ioctl 命令：_IOR = 从内核读 */
#define IOCTL_EXAMPLE _IOR('a', 1, int32_t *)

/* unlocked_ioctl 回调 */
static long dev_ioctl(struct file *filep, unsigned int cmd, unsigned long arg)
{
    switch (cmd) {
```

```
        case IOCTL_EXAMPLE:
            /* 将内核变量 value 的值拷到用户空间 arg 指向的地址 */
            if (copy_to_user((int32_t *)arg, &value, sizeof(value)))
                return -EACCES;
            break;
        }
        return 0;
    }

    /* 注册设备 + 自动创建设备节点 */
    majorNumber = register_chrdev(0, DEVICE_NAME, &fops);
    ioctlClass = class_create(THIS_MODULE, CLASS_NAME);
    ioctlDevice = device_create(ioctlClass, NULL,
                                MKDEV(majorNumber, 0), NULL, DEVICE_NAME);
    /* 自动生成 /dev/ioctl_example，无需 mknod */
```

- `_IOR('a', 1, ...)`: magic='a', cmd=1, 读方向
- `device_create` 自动创建设备节点，不需要手动 `mknod`

6.4 关键API

`register_chrdev(0, name, &fops)`

```
int register_chrdev(unsigned int major, const char *name,
                    const struct file_operations *fops);
```

- **major=0**：让内核自动分配一个空闲主设备号
- **name**：设备名，出现在 `/proc/devices` 中
- **fops**：文件操作表，至少包含 `.open` 和 `.unlocked_ioctl`
- **返回**：分配的主设备号（正数），失败返回负数

`class_create` + `device_create`

```
struct class *class_create(struct module *owner, const char *name);
struct device *device_create(struct class *cls, struct device *parent,
                             dev_t devt, void *drvdata, const char *fmt, ...);
```

- 配合使用后**自动在 `/dev/` 下创建设备节点**（`/dev/ioctl_example`），无需手动 `mknod`
- `device_create` 的 `devt = MKDEV(major, 0)`

`_IOR(magic, cmd, type)`

```
#define IOCTL_EXAMPLE _IOR('a', 1, int32_t *)
```

- **direction**（`_IOR`/`_IOW`/`_IOWR`）：数据方向——从内核读 / 向内核写 / 双向

- **magic**：魔数，选一个不冲突的字符（`'a'~'z'`），用于防止 ioctl 命令码冲突
- **cmd**：命令编号（`1`），同一个设备的不同命令用不同编号
- **type**：参数类型，编译器用这个做类型检查

`unlocked_ioctl(file, cmd, arg)`

```
long (*unlocked_ioctl)(struct file *file, unsigned int cmd, unsigned long arg);
```

- **cmd**：`_IOR` 等宏生成的实际命令码
- **arg**：用户传入的指针（`unsigned long` 类型），必须转型后才能用

6.5 数据流

```
用户态                                    内核态
──────                                    ──────
fd = open("/dev/ioctl_example")           dev_open() → return 0
  │
ioctl(fd, IOCTL_EXAMPLE, &value)
  │
  ▼  ioctl() 系统调用
  │──────────────────────────►            dev_ioctl(file, cmd=IOCTL_EXAMPLE, arg)
                                           │
                                           ├─ copy_to_user(arg, &value, 4)
                                           │  value = 0
                                           └─ return 0
  ◄──────────────────────────
  │
printf("Received value from kernel: 0")
close(fd)                                 dev_release() → return 0
```

6.6 编译与验证

```
root@ubuntu:/home/tjn/commu# mkdir 04-ioctl
root@ubuntu:/home/tjn/commu# cd 04-ioctl/
root@ubuntu:/home/tjn/commu/04-ioctl# vim Makefile
root@ubuntu:/home/tjn/commu/04-ioctl# vim ioctl_user.c
root@ubuntu:/home/tjn/commu/04-ioctl# vim ioctl_example.c
root@ubuntu:/home/tjn/commu/04-ioctl# make
make -C /lib/modules/4.15.0-213-generic/build M=/home/tjn/commu/04-ioctl modules
make[1]: Entering directory '/usr/src/linux-headers-4.15.0-213-generic'
  CC [M]  /home/tjn/commu/04-ioctl/ioctl_example.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC      /home/tjn/commu/04-ioctl/ioctl_example.mod.o
  LD [M]  /home/tjn/commu/04-ioctl/ioctl_example.ko
make[1]: Leaving directory '/usr/src/linux-headers-4.15.0-213-generic'
root@ubuntu:/home/tjn/commu/04-ioctl# gcc -o ioctl_user ioctl_user.c
```

```
root@ubuntu:/home/tjn/commu/04-ioctl# insmod ioctl_example.ko
root@ubuntu:/home/tjn/commu/04-ioctl# ./ioctl_user
Received value from kernel: 0
root@ubuntu:/home/tjn/commu/04-ioctl# rmmod ioctl_example
```

验证结果：`ioctl_user` 通过 `ioctl()` 系统调用从内核读到值 `0`（内核模块中 `value` 初始化为 0），证明 ioctl 通信管道正常。

---

7. 05-字符设备

7.1 通信原理

字符设备按字节流传输，用户态通过标准文件操作（`open`/`read`/`write`/`close`）与内核驱动交互。内核模块实现 `file_operations` 结构体中的对应回调函数，通过 `copy_from_user` / `copy_to_user` 在内核和用户空间之间搬移数据。

7.2 文件结构

```
05-字符设备/
├── Makefile                # obj-m += char_device_example.o
└── char_device_example.c   # 内核模块
```

7.3 关键代码解析

```c
/* write: 用户数据 → 内核 */
static ssize_t dev_write(struct file *filep, const char __user *buffer,
                         size_t len, loff_t *offset)
{
    if (copy_from_user(message, buffer, len))
        return -EFAULT;
    message[len] = '\0';
    messageSize = strlen(message);
    return len;
}

/* read: 内核数据 → 用户 */
static ssize_t dev_read(struct file *filep, char __user *buffer,
                        size_t len, loff_t *offset)
{
    if (*offset > 0)
        return 0;                       /* EOF */
    if (copy_to_user(buffer, message, messageSize))
        return -EFAULT;
    *offset += messageSize;
    return messageSize;
}
```

```
static struct file_operations fops = {
    .open    = dev_open,
    .read    = dev_read,
    .write   = dev_write,
    .release = dev_release,
};
```

- 利用 *offset 控制读取位置，防止 cat 循环读取
- device_create 自动生成 /dev/char_example 节点

7.4 关键API

copy_from_user(to, from, n)

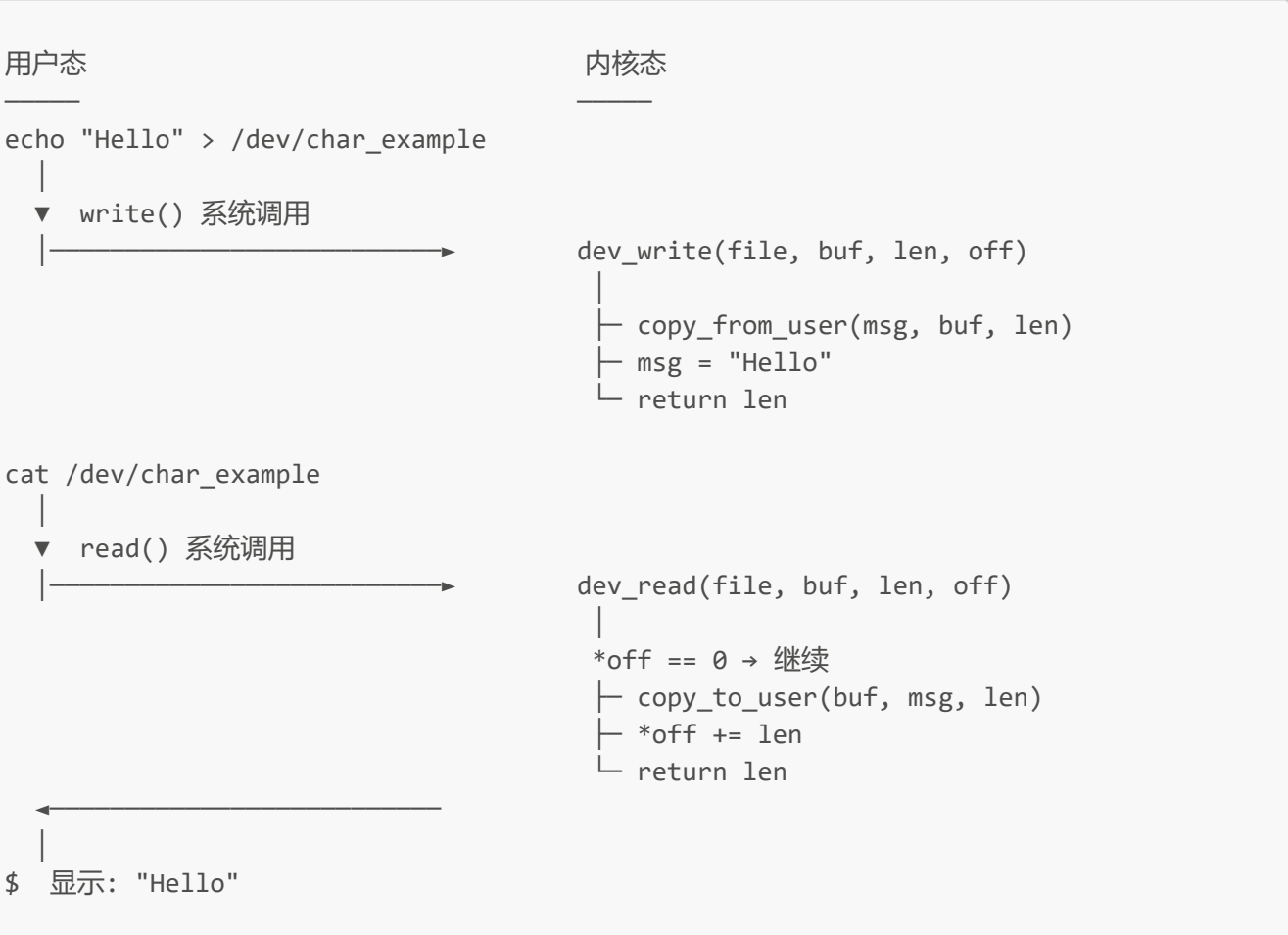
```
unsigned long copy_from_user(void *to, const void __user *from, unsigned long n);
```

- **作用**：用户空间 → 内核空间数据拷贝。和 copy_to_user 对称，返回值含义相同（0=成功）
- 设备驱动中 write 回调必须用它来读取用户写入的数据

register_chrdev / class_create / device_create

- 同 04-ioctl 关键API，字符设备注册的标准三步

7.5 数据流



7.6 编译与验证

```
root@ubuntu:/home/tjn/commu# mkdir 05-character
root@ubuntu:/home/tjn/commu# cd 05-character/
root@ubuntu:/home/tjn/commu/05-character# vim Makefile
root@ubuntu:/home/tjn/commu/05-character# vim char_device_example.c
root@ubuntu:/home/tjn/commu/05-character# make
make -C /lib/modules/4.15.0-213-generic/build M=/home/tjn/commu/05-character
modules
make[1]: Entering directory '/usr/src/linux-headers-4.15.0-213-generic'
  CC [M]  /home/tjn/commu/05-character/char_device_example.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC      /home/tjn/commu/05-character/char_device_example.mod.o
  LD [M]  /home/tjn/commu/05-character/char_device_example.ko
make[1]: Leaving directory '/usr/src/linux-headers-4.15.0-213-generic'
root@ubuntu:/home/tjn/commu/05-character# insmod char_device_example.ko
root@ubuntu:/home/tjn/commu/05-character# echo "Hello, Kernel!" >
/dev/char_example
root@ubuntu:/home/tjn/commu/05-character# cat /dev/char_example
Hello, Kernel!
root@ubuntu:/home/tjn/commu/05-character# rmmod char_device_example
```

验证结果：`echo` 写入 → `cat` 读出 `"Hello, Kernel!"`，字符设备 read/write 通路正常。

---

8. 06-块设备

8.1 通信原理

块设备以固定大小的扇区为单位传输数据。内核模块通过 `register_blkdev` 注册、`blk_init_queue` 创建请求队列，`gendisk` 表示具体的块设备。用户态用 `dd` 以指定块大小读写。

8.2 文件结构

```
06-块设备/
├── Makefile                  # obj-m += block_device_example.o
└── block_device_example.c    # 内核模块
```

8.3 关键代码解析

```
#define DEVICE_SECTORS 1024
#define SECTOR_SIZE 512

/* 分配扇区数据内存（vzalloc 零初始化，避免乱码） */
device_data = vzalloc(DEVICE_SECTORS * SECTOR_SIZE);
```

```
    /* 注册块设备 */
    major_num = register_blkdev(major_num, DEVICE_NAME);
    queue = blk_init_queue(block_example_request, NULL);
    gd = alloc_disk(1);
    gd->major = major_num;
    gd->fops  = &block_example_ops;
    gd->queue = queue;
    set_capacity(gd, DEVICE_SECTORS);    /* 1024 扇区 → 512KB */
    add_disk(gd);

    /* 处理每个 I/O 请求 */
    static void block_example_request(struct request_queue *q)
    {
        req = blk_fetch_request(q);
        rq_for_each_segment(bvec, req, iter) {
            buffer = page_address(bvec.bv_page) + bvec.bv_offset;
            if (rq_data_dir(req) == WRITE)
                memcpy(device_data + offset, buffer, bvec.bv_len); /* 写 */
            else
                memcpy(buffer, device_data + offset, bvec.bv_len); /* 读 */
        }
        __blk_end_request_all(req, 0);
    }
```

- `blk_init_queue` + `blk_fetch_request` 是 4.15 内核的单队列 API（5.x 已迁移到 blk-mq）
- `add_disk(gd)` 自动生成 `/dev/block_example` 节点

8.4 关键API

`register_blkdev(major, name)`

```
int register_blkdev(unsigned int major, const char *name);
```

- **major=0**：动态分配主设备号
- 返回分配的主设备号，≤0 表示失败

`blk_init_queue(request_fn, lock)`

```
struct request_queue *blk_init_queue(request_fn_proc *rfn, spinlock_t *lock);
```

- **rfn**：请求处理回调函数，参数是 `struct request_queue *`
- **lock=NULL**：使用内核默认的自旋锁
- Ubuntu 18.04 内核 4.15 仍支持此 API（5.x 已迁移到 blk-mq）

`alloc_disk(minors)` + `add_disk(gd)`

```

```

```
struct gendisk *alloc_disk(int minors);
void add_disk(struct gendisk *gd);
```

- **minors=1**：只有一个分区
- `add_disk` 后自动在 `/dev/` 下生成设备节点

`blk_fetch_request(q)` / `__blk_end_request_all(req, err)`

```
struct request *blk_fetch_request(struct request_queue *q);
bool __blk_end_request_all(struct request *req, int error);
```

- `blk_fetch_request`：从请求队列取出下一个 I/O 请求（循环调用直到返回 NULL）
- `__blk_end_request_all`：标记该请求处理完成，`error=0` 表示成功

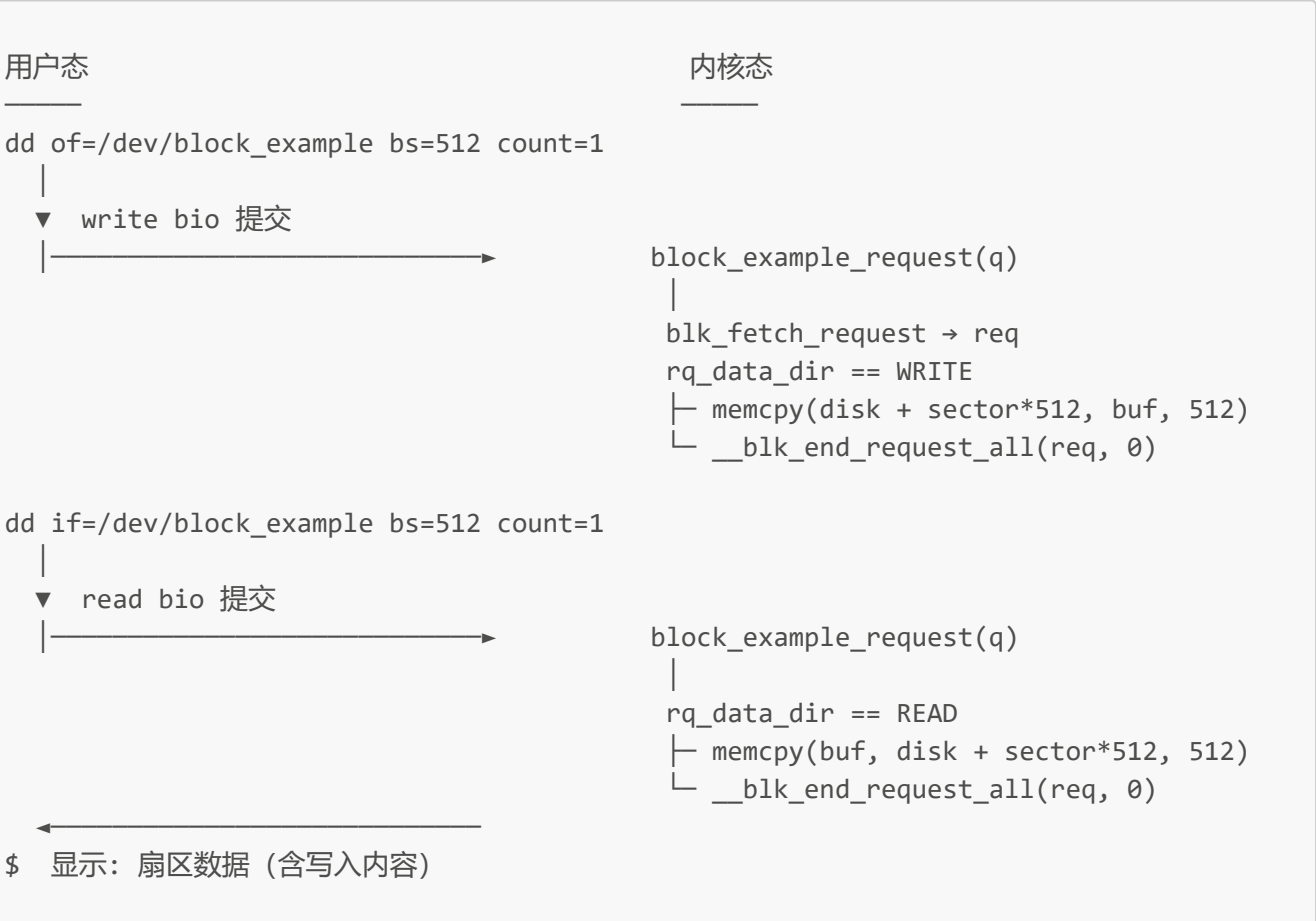
`rq_for_each_segment(bvec, req, iter)`

- 遍历 bio 请求中的每个内存段（segment），`bvec.bv_page` 是数据所在的内存页

`vzalloc(size)` **vs** `vmalloc(size)`

- `vmalloc` 不保证清零 → 未写入区域是随机垃圾数据
- `vzalloc` 等价于 `vmalloc` + `memset(0)` → 扇区未写部分是 `\0`

8.5 数据流



8.6 编译与验证

```
root@ubuntu:/home/tjn/commu/06-block# insmod block_device_example.ko
root@ubuntu:/home/tjn/commu/06-block# echo "Hello Block" | dd
of=/dev/block_example bs=512 count=1
0+1 records in
0+1 records out
12 bytes copied, 0.000266324 s, 45.1 kB/s
root@ubuntu:/home/tjn/commu/06-block# dd if=/dev/block_example bs=512 count=1 |
head -c 20
1+0 records in
1+0 records out
512 bytes copied, 8.3462e-05 s, 6.1 MB/s
Hello Block
root@ubuntu:/home/tjn/commu/06-block# rmmod block_device_example
root@ubuntu:/home/tjn/commu/06-block#
```

验证结果：`dd` 写入 → `dd` 读出 `"Hello Block"`，块设备扇区读写通路正常。

8.7 遇到的问题

问题：`dd` 读取时 `Hello Block` 后跟大量乱码，且每次显示不同。

原因：`vmalloc` 分配的内存不保证清零，未写入的扇区空间保留随机内核内存数据。

解决：将 `vmalloc` 改为 `vzalloc`（零初始化）。写入 12 字节后扇区剩余字节为 `\0`，不再产生乱码。

---

9. 07-mmap内存映射

9.1 通信原理

`mmap` 将内核分配的物理内存直接映射到用户进程的虚拟地址空间，实现零拷贝共享。内核模块中调用 `remap_pfn_range()` 建立页表映射；用户态用 `mmap()` 获取映射地址，之后就能像访问本地内存一样直接读写内核数据，无需 `read`/`write` 系统调用。

9.2 文件结构

```
07-mmap内存映射/
├── Makefile               # obj-m += mmap_kernel_example.o
├── mmap_kernel_example.c  # 内核模块
└── mmap_user_example.c    # 用户态程序
```

9.3 关键代码解析

内核模块：

```
static char *kernel_buffer;

/* 分配内核内存，写入初始数据 */
kernel_buffer = kmalloc(MEM_SIZE, GFP_KERNEL);
strcpy(kernel_buffer, "Hello from kernel space!");

/* mmap 回调：将内核内存映射到用户空间 */
static int mmap_mmap(struct file *filep, struct vm_area_struct *vma)
{
    /* 将内核虚拟地址转为物理页帧号 */
    unsigned long pfn = virt_to_phys(kernel_buffer) >> PAGE_SHIFT;

    /* 建立用户虚拟地址 → 内核物理地址的页表映射 */
    if (remap_pfn_range(vma, vma->vm_start, pfn,
                        vma->vm_end - vma->vm_start,
                        vma->vm_page_prot))
        return -EAGAIN;
    return 0;
}
```

用户态:

```
fd = open("/dev/mmap_example", O_RDWR);

/* 将 4096 字节内核内存映射到本地地址空间 */
mapped_mem = mmap(NULL, MEM_SIZE, PROT_READ | PROT_WRITE,
                  MAP_SHARED, fd, 0);

/* 直接读—不用 read() 系统调用 */
printf("Data from kernel space: %s\n", mapped_mem);

/* 直接写—不用 write() 系统调用 */
sprintf(mapped_mem, "Hello from user space!");
```

9.4 关键API

`kmalloc(size, GFP_KERNEL)`

```
void *kmalloc(size_t size, gfp_t flags);
```

- 分配连续内核内存。`GFP_KERNEL` 是最常用标志（可能会睡眠，不能用于中断上下文）
- 用完后必须 `kfree()`

`virt_to_phys(vaddr)`

```
phys_addr_t virt_to_phys(volatile void *address);
```

- 将内核虚拟地址（线性映射区）转为物理地址
- `>> PAGE_SHIFT` 得到物理页帧号（PFN），传给 `remap_pfn_range`

`remap_pfn_range(vma, vma->vm_start, pfn, size, prot)`

```
int remap_pfn_range(struct vm_area_struct *vma, unsigned long addr,
                    unsigned long pfn, unsigned long size, pgprot_t prot);
```

- **作用**：建立用户空间虚拟地址到物理页帧的页表映射
- **vma**：内核在调用 `.mmap` 回调时自动传入
- **pfn**：物理页帧号（`virt_to_phys(buf) >> PAGE_SHIFT`）
- **size**：`vma->vm_end - vma->vm_start`
- 返回 0 表示成功

`mmap(NULL, len, prot, flags, fd, offset)` **（用户态）**

```
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

- **addr=NULL**：让内核选择映射地址
- **prot=PROT_READ|PROT_WRITE**：可读可写
- **flags=MAP_SHARED**：修改对内核可见（反之 `MAP_PRIVATE` 只在本进程生效）
- 映射后的地址可直接指针访问，**无需** `read`/`write` 系统调用

9.5 数据流

```
用户态                                              内核态
──────                                              ──────
fd = open("/dev/mmap_example")
  │
mmap(NULL, 4096, RW, SHARED, fd, 0)
  │  ↕ 页表直接映射到同一块物理内存
  │  ↕ 读写不再经过系统调用!
  │
mapped_mem = 0x7f...          ←──→          kernel_buffer = 0xffff...
    (用户虚拟地址)                              (内核虚拟地址)
                                  │
                                  │
                virt_to_phys()    ↓
                        ┌── 同一块物理页帧 ──┐
                        │    0x1a2b3000      │
                        └────────────────────┘

printf("%s", mapped_mem)       →  读到 "Hello from kernel space!"
sprintf(mapped_mem, "Hi!")     →  直接写内核内存，无需 write()
```

9.6 编译与验证

```
root@ubuntu:/home/tjn/commu/07-mmap# insmod mmap_kernel_example.ko
root@ubuntu:/home/tjn/commu/07-mmap# ./mmap_user
Data from kernel space: Hello from kernel space!
root@ubuntu:/home/tjn/commu/07-mmap# rmmod mmap_kernel_example
```

验证结果：用户态通过 `mmap` 直接读到内核写入的 `"Hello from kernel space!"`，零拷贝通信成功。

---

10. 08-信号（Signals）

10.1 通信原理

内核模块通过 `send_sig_info()` 向指定 PID 的用户进程发送信号，可附带一个 `int` 类型的数据（`si_int`）。用户态用 `sigaction()` 注册信号处理函数，在 `SA_SIGINFO` 模式下能接收附带的数据。信号是异步的内核→用户态通知机制。

10.2 文件结构

```
08-信号/
├── Makefile                  # obj-m += signal_kernel_example.o
├── signal_kernel_example.c   # 内核模块（内核线程循环发信号）
└── signal_user_example.c     # 用户态程序（接收信号）
```

10.3 关键代码解析

内核模块：

```
#define SIGNAL_NUM 44   /* 实时信号范围 SIGRTMIN~SIGRTMAX */

/* insmod 时通过 pid=xxx 参数传入目标进程 PID */
module_param(pid, int, 0644);

/* 填充信号附加数据 */
memset(&info, 0, sizeof(struct siginfo));
info.si_signo = SIGNAL_NUM;
info.si_code  = SI_QUEUE;     /* 用户态可通过 sigqueue 发送 */
info.si_int   = 1234;         /* 附带 int 数据 */

/* 每 5 秒向目标进程发送一次 */
while (!kthread_should_stop()) {
    send_sig_info(SIGNAL_NUM, &info, task_struct);
    ssleep(5);
}
```

用户态:

```
static void signal_handler(int signo, siginfo_t *info, void *extra)
{
    printf("Received signal %d with value %d\n", signo, info->si_int);
    /* 输出: Received signal 44 with value 1234 */
}

/* 注册 SA_SIGINFO 模式的信号处理 */
action.sa_flags     = SA_SIGINFO;
action.sa_sigaction = signal_handler;
sigaction(44, &action, NULL);

while (1) {
    pause();  /* 阻塞等待信号 */
}
```

10.4 关键API

module_param(name, type, perm)

```
module_param(pid, int, 0644);
```

- 声明模块参数，`insmod xxx.ko pid=12345` 时传入
- `perm=0644` 会在 `/sys/module/xxx/parameters/pid` 创建对应的 sysfs 文件

kthread_run(threadfn, data, name)

```
struct task_struct *kthread_run(int (*threadfn)(void *data), void *data,
                                const char namefmt[], ...);
```

- 创建并启动一个内核线程，`threadfn` 在线程上下文中执行
- 返回的 `task_struct` 指针可用于 `kthread_stop()` 停止线程

find_get_pid(pid) + pid_task(pid_struct, PIDTYPE_PID)

```
struct pid *find_get_pid(pid_t nr);
struct task_struct *pid_task(struct pid *pid, enum pid_type type);
```

- 通过 PID 号找到对应的 `task_struct`（进程控制块）

send_sig_info(sig, &info, task)

```
int send_sig_info(int sig, struct siginfo *info, struct task_struct *p);
```

- **sig**：信号编号。自定义信号建议用 `SIGRTMIN+1` ~ `SIGRTMAX`（实时信号，带数据），如 44
- **info.si_signo**：信号编号
- **info.si_code=SI_QUEUE**：标记为"带数据的实时信号"
- **info.si_int**：附带传给用户态的 int 数据

`sigaction(sig, &action, NULL)` **（用户态）**

```
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
```

- **sa_flags=SA_SIGINFO**：使用三参数处理函数（`handler(int, siginfo_t*, void*)`），这样才能收到 `si_int`
- **sa_sigaction**：信号处理函数指针
- `pause()` 阻塞等待任意信号

10.5 数据流

```
用户态                                          内核态
————                                            ————
./signal_user (PID=11459)
  sigaction(44, handler)            insmod signal_kernel_example pid=12345
  pause() 阻塞等待                     │
      ▲                               ├─ 找到 PID=11459 的 task_struct
      │                               ├─ 每 5 秒:
      │    signal 44                  ├─ send_sig_info(44, {si_int=1234})
      │  ◄──────────────────────────  ┘
      │
  signal_handler(44, info)
    printf("Received signal 44 with value 1234")
  pause() 继续等待...
```

10.6 编译与验证

> 以下为双终端操作截图：

终端一：

```
root@ubuntu:/home/tjn/commu/08-signal# make
make -C /lib/modules/4.15.0-213-generic/build M=/home/tjn/commu/08-signal modules
make[1]: Entering directory '/usr/src/linux-headers-4.15.0-213-generic'
  CC [M]  /home/tjn/commu/08-signal/signal_kernel_example.o
  Building modules, stage 2.
  MODPOST 1 modules
  CC      /home/tjn/commu/08-signal/signal_kernel_example.mod.o
  LD [M]  /home/tjn/commu/08-signal/signal_kernel_example.ko
make[1]: Leaving directory '/usr/src/linux-headers-4.15.0-213-generic'
root@ubuntu:/home/tjn/commu/08-signal# gcc -o signal_user signal_user_example.c
root@ubuntu:/home/tjn/commu/08-signal# ./signal_user
Waiting for signals... (PID: 11459)
Received signal 44 with value 1234
Received signal 44 with value 1234
Received signal 44 with value 1234
```

终端二：

```
root@ubuntu:~# cd /home/tjn/commu/08-signal/
root@ubuntu:/home/tjn/commu/08-signal# insmod signal_kernel_example.ko pid=11459
root@ubuntu:/home/tjn/commu/08-signal#
```

操作流程：

1. 终端一：编译 + 运行 `./signal_user`，程序打印 `Waiting for signals... (PID: xxx)`
2. 终端二：`insmod signal_kernel_example.ko pid=xxx`（传入终端一的 PID）
3. 终端一每 5 秒输出 `Received signal 44 with value 1234`
4. 终端二：`rmmod signal_kernel_example` 停止

验证结果：内核模块通过信号向用户态进程推送了 `int` 值 `1234`，异步通知机制正常。

---

11. 09-inotify和fanotify

11.1 通信原理

`inotify` 和 `fanotify` 是内核内置的文件系统事件监控子系统。用户态通过系统调用（`inotify_init()` / `fanotify_init()`）创建监控实例，通过 `inotify_add_watch()` / `fanotify_mark()` 注册监控目标，内核 VFS 层在文件操作发生时将事件写入 fd 对应的内核缓冲区，用户态 `read()` 读取。不需要自定义内核模块——通信管道由内核提供。

11.2 文件结构

```
09-inotify和fanotify/
├── inotify_example.c    # inotify 监控程序
└── fanotify_example.c   # fanotify 监控程序
```

11.3 关键代码解析

inotify：

```
fd = inotify_init();                    /* 创建 inotify 实例 */

/* 监控 /tmp 下的创建、删除、修改事件 */
wd = inotify_add_watch(fd, "/tmp", IN_CREATE | IN_DELETE | IN_MODIFY);

/* 阻塞读取事件 */
while ((length = read(fd, buffer, EVENT_BUF_LEN)) > 0) {
    struct inotify_event *event = (struct inotify_event *)&buffer[i];
    if (event->mask & IN_CREATE)
        printf("The file %s was created.\n", event->name);
    if (event->mask & IN_DELETE)
        printf("The file %s was deleted.\n", event->name);
    if (event->mask & IN_MODIFY)
        printf("The file %s was modified.\n", event->name);
}
```

fanotify:

```
/* FAN_CLASS_NOTIF: 仅通知，不阻止操作；O_RDONLY: 只读 */
fanotify_fd = fanotify_init(FAN_CLOEXEC | FAN_CLASS_NOTIF, O_RDONLY);

/* 监控 /tmp 挂载点下所有文件的打开和关闭 */
fanotify_mark(fanotify_fd, FAN_MARK_ADD | FAN_MARK_MOUNT,
              FAN_OPEN | FAN_CLOSE, AT_FDCWD, "/tmp");

/* poll 等待 + read 读取事件 */
while (poll(fds, 1, -1) > 0) {
    length = read(fanotify_fd, buffer, EVENT_BUF_LEN);
    /* 遍历事件 */
}
```

- `inotify` 适合监控指定目录/文件的具体事件，轻量易用
- `fanotify` 可以监控整个挂载点，支持更多事件类型（需 root 权限）

11.4 关键API

inotify

`inotify_init()`

```
int inotify_init(void);
```

- 创建 inotify 实例，返回文件描述符。之后用 `read(fd, ...)` 读取事件

`inotify_add_watch(fd, path, mask)`

```
int inotify_add_watch(int fd, const char *pathname, uint32_t mask);
```

- **pathname**：要监控的目录或文件路径
- **mask**：事件类型位掩码，OR 组合 `IN_CREATE | IN_DELETE | IN_MODIFY` 等
- 返回 watch descriptor（`wd`），用于 `inotify_rm_watch`

`inotify_rm_watch(fd, wd)` — 移除一个监控目标

fanotify

`fanotify_init(flags, event_f_flags)`

```
int fanotify_init(unsigned int flags, unsigned int event_f_flags);
```

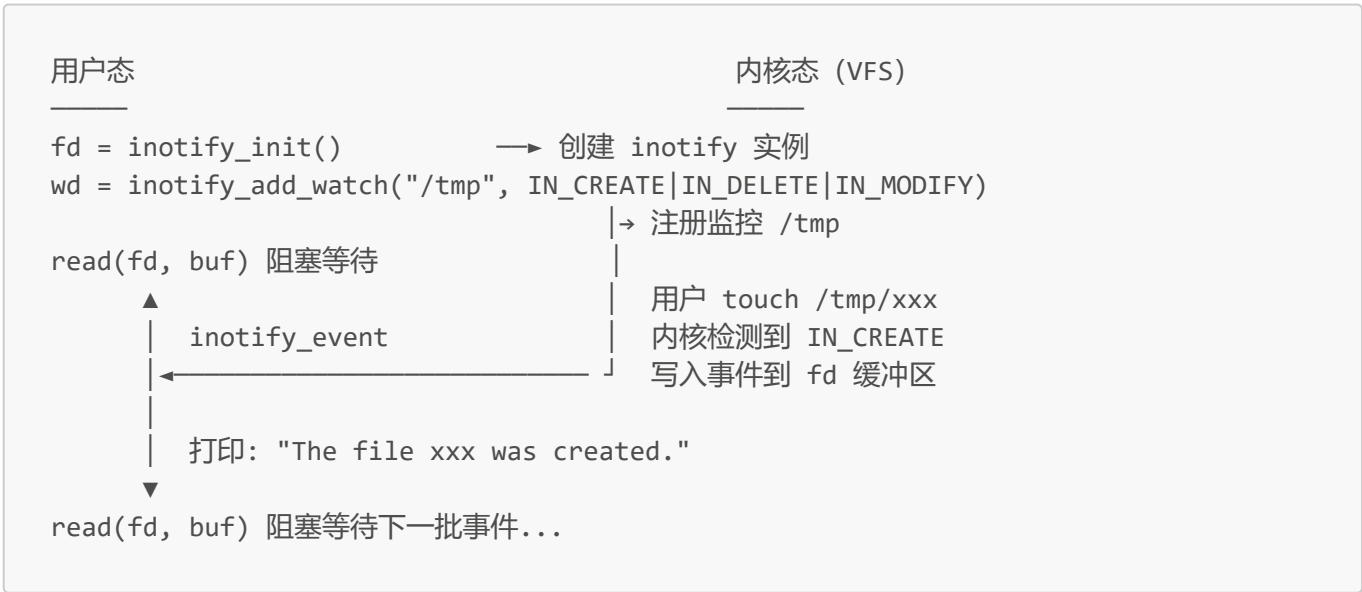
- **flags=FAN_CLOEXEC|FAN_CLASS_NOTIF**：自动 close-on-exec + 仅通知模式（不阻止操作）
- 返回的文件描述符用 `read()` 读取事件，建议搭配 `poll()` 使用

`fanotify_mark(fd, flags, mask, dirfd, path)`

```
int fanotify_mark(int fanotify_fd, unsigned int flags, uint64_t mask,
                  int dirfd, const char *pathname);
```

- **flags=FAN_MARK_ADD|FAN_MARK_MOUNT**：添加监控整个挂载点
- **mask=FAN_OPEN|FAN_CLOSE**：监控文件打开和关闭事件
- **dirfd=AT_FDCWD**：路径相对于当前工作目录

11.5 数据流



11.6 编译与验证

```
【终端1 - 运行监控程序】
root@ubuntu:/home/tjn/commu/08-signal# ./inotify
Monitoring /tmp for changes...
The file sig_vrfy_a.txt was created.
The file sig_vrfy_a.txt was modified.
The file sig_vrfy_a.txt was modified.
The file sig_vrfy_a.txt was deleted.
The file sig_vrfy_b.txt was created.
The file sig_vrfy_b.txt was deleted.

【终端2 - 触发文件事件（另一终端）】
root@ubuntu:/home/tjn/commu/08-signal# echo 'first line' > /tmp/sig_vrfy_a.txt
root@ubuntu:/home/tjn/commu/08-signal# echo 'append' >> /tmp/sig_vrfy_a.txt
root@ubuntu:/home/tjn/commu/08-signal# rm -f /tmp/sig_vrfy_a.txt
root@ubuntu:/home/tjn/commu/08-signal# touch /tmp/sig_vrfy_b.txt
root@ubuntu:/home/tjn/commu/08-signal# rm -f /tmp/sig_vrfy_b.txt

终端1 在终端2 执行上述命令的同时，实时捕获到对应事件
（创建/2次修改/删除/创建/删除，共 6 个，与命令一一对应）。
```

验证结果：终端1实时捕获到终端2的 6 次文件操作（创建→修改×2→删除→创建→删除），事件与命令一一对应。内核通过系统调用将 VFS 事件推送给用户态。

---

12. 命令速查表

操作	命令
安装内核编译依赖	`sudo apt install build-essential linux-headers-$(uname -r)`
编译内核模块	`make`
清理编译产物	`make clean`
加载内核模块	`insmod xxx.ko`
卸载内核模块	`rmmod xxx`
查看已加载模块	`lsmod`
查看内核日志	`dmesg | tail`
编译用户态程序	`gcc -o xxx xxx.c`（Netlink/mmap/ioctl 无额外链接）
读取 proc 文件	`cat /proc/helloworld`
读写 sysfs 文件	`cat /sys/kernel/my_sysfs_example` / `echo xxx >`
字符设备读写	`echo xxx > /dev/char_example` / `cat /dev/char_example`
块设备读写	`dd of=/dev/block_example bs=512 count=1` / `dd if=`
ioctl 测试	`./ioctl_user`
mmap 测试	`./mmap_user`

操作	命令
信号测试	终端1: `./signal_user` → 终端2: `insmod signal_kernel_example.ko pid=xxx`
inotify 监控	`./inotify`
fanotify 监控	`./fanotify`